2.2 Research Gap

The literature shows that coding agents can now make repository-scale changes [2], [3], [4], that their code and actions are risky in measurable ways [5], [6], [7], [8], that container security depends on configuration [11], [12], [13], and that governance research recommends human approval and legible logs [9], [10]. Agent vendors have since shipped security features of their own[1]: OS-level sandboxes for the commands an agent runs in Claude Code and the OpenAI Codex CLI, a firewalled hosted environment for the GitHub Copilot cloud agent, local microVM sandboxes in Docker Sandboxes, and a Docker runtime with an event stream in OpenHands. Table 2.2 compares them with VaultShield, based on their official documentation.

Table 2.2: Comparison with existing AI coding-agent security solutions

Control	Claude Code	Codex CLI	Copilot cloud agent	Docker Sandboxes	OpenHands	VaultShield
Agent compatibility	Claude Code only	Codex only	Copilot only	Several agents	Its own agents	Any agent with an adapter (four built)
Isolation approach	OS sandbox for shell commands	OS sandbox	Hosted ephemeral environment	Local microVM	Docker container	Docker container per run (shared kernel)
Real .env kept from agent	–	–	Only copilot-environment secrets	–	–	Shape-preserving dummy values
Host checkout protected	–	–	Yes (hosted)	–	Configurable	Fresh clone; host never mounted
Network restricted by default	Configurable	Yes	Yes (firewall)	Configurable	–	Opt-in allowlist proxy
Monitoring and audit evidence	–	–	Session logs	–	Event stream	Event log, egress log, screenshots
Change record outside agent's control	–	–	–	–	–	Separate capture helper
Human approval before repository submission	Per-action prompts	Per-action approval policy	Review after the branch is pushed	–	Per-action confirmation	Whole change held until approved

"–" means the cited documentation does not describe the control. The comparison exposes four gaps. First, each sandbox protects only its own vendor's agent, with no common policy or evidence format. Second, the project's .env sits inside the workspace that local sandboxes allow, and deny rules proved brittle in our spike (Section 7.6). Third, any activity record is produced beside the agent, which could tamper with Git state to misreport its change. Fourth, approval is per action or comes after the change is pushed. Existing products lead on isolation strength and default-deny networking. VaultShield closes the four gaps together for any agent on the developer's machine: dummy credentials and a one-shot clone token keep real secrets away, a separate capture helper builds the change record, and an approval gate holds the change until it is approved.

[1]Vendor documentation, accessed 1 Nov 2026: code.claude.com/docs/en/sandboxing; developers.openai.com/codex/sandbox; docs.github.com (Copilot cloud agent); docs.docker.com/ai/sandboxes; docs.all-hands.dev.

Chapter 3

Project Design

3.1 Existing System

Developers currently run AI coding agents in one of three ways. The most common is raw local execution, where the agent's CLI or editor extension runs in the developer's own checkout with the developer's environment, permissions and network access, and its changes appear as uncommitted edits. Some products add permission prompts or per-tool deny rules, but these are product-specific and often relaxed by users who find repeated prompts tedious. The second practice is ad-hoc containerisation, which usually bind-mounts the real working tree, so every write still lands on the host and the real .env file travels into the container. The third, development containers and cloud workspaces, is designed for convenience rather than for constraining an untrusted actor and holds nothing for review. Figure 3.1 summarises this workflow.

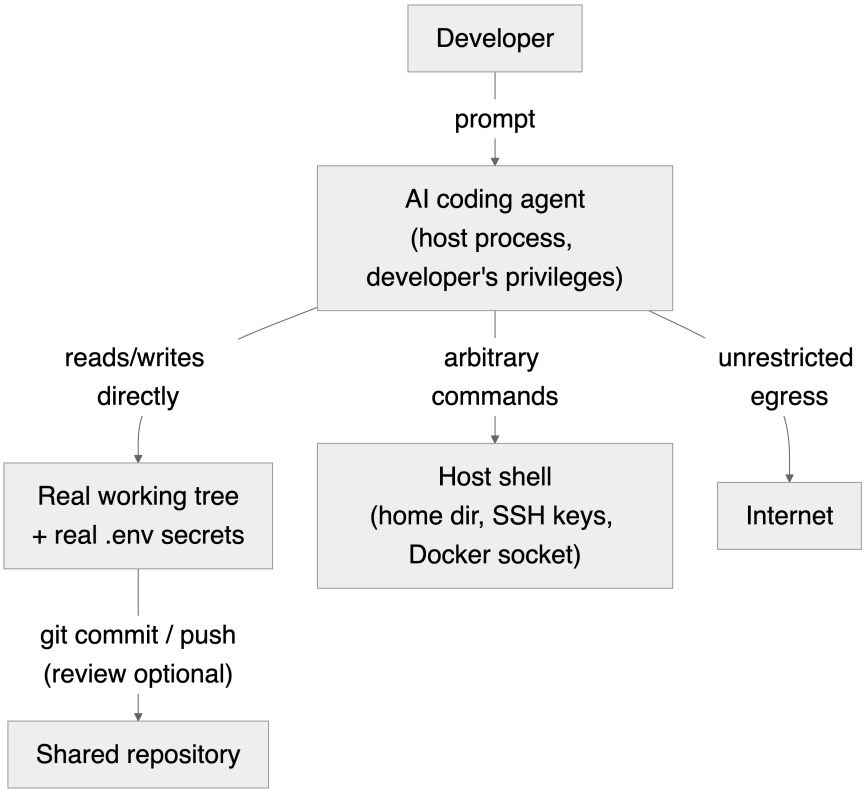


Figure 3.1: Existing workflow – direct agent execution on the developer's host

As Figure 3.1 shows, execution and acceptance are a single step and every arrow out of the agent touches a real asset. There is no isolation of side effects, no separation between the agent and the developer's secrets, no structured record of what happened, no enforced review before changes are committed, no automatic cleanup of processes the agent started,

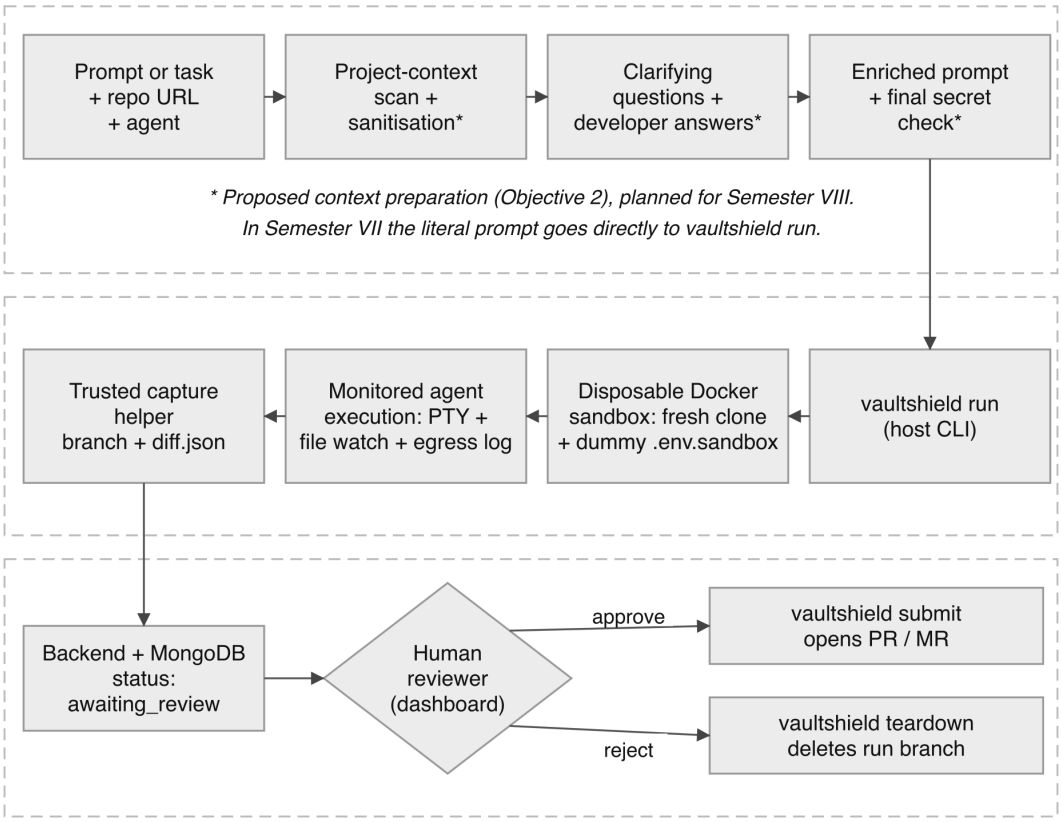


Figure 3.2: Proposed VaultShield pipeline

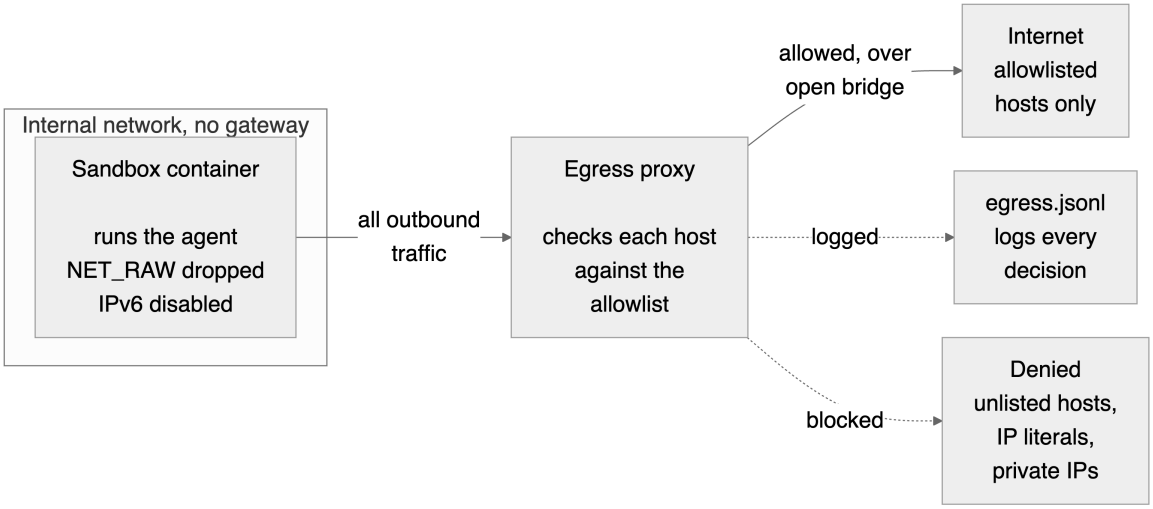


Figure 3.3: Per-run egress network topology

it kills the sandbox and extracts a filtered archive of the work tree. A fresh capture helper container with no network, no capabilities and no environment rebuilds the repository from the trusted base, refuses unsafe Git configuration, commits every change to the run branch as exactly one commit, and writes diff.json with per-file additions, deletions and size-capped patches. The branch is exported as a Git bundle because the helper is destroyed at the end of the run.

3.3.4 Review and Human Approval Service

The Run.status field is a state machine, shown in Figure 3.4. The CLI may move a run from running to awaiting_review or failed, and the approve and reject endpoints change the status only if it is still awaiting_review, inside a MongoDB transaction that also creates the Approval record. Of two concurrent decisions exactly one succeeds, and the other receives HTTP 409 with the run's actual status.

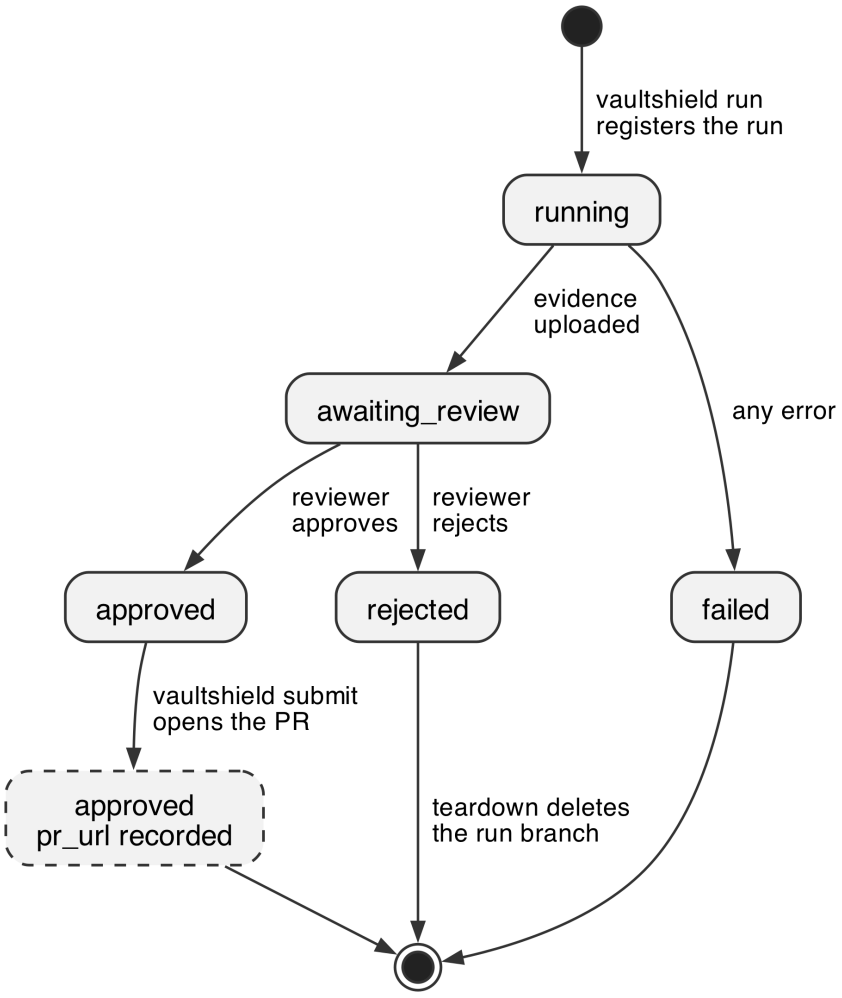


Figure 3.4: Run status state machine

vaultshield submit re-reads the run and refuses before any side effect unless its status is approved. For an approved run it checks that the current checkout matches the original repository, imports the branch from the verified bundle, pushes it without force, opens the pull or merge request and records its URL on the run. vaultshield teardown deletes the branch of a rejected run.

3.3.5 Backend API and Dashboard

The backend is a Django 5.2 project with Django REST Framework and MongoDB as its system of record. Every API path except login requires token authentication. The CLI registers runs, uploads events, diffs and screenshots, and patches the status; the dashboard reads run details and records decisions. Celery workers with Redis run a static security scan over the added lines of each diff, reap orphaned sandboxes every 15 minutes and flag approved but unsubmitted runs. The dashboard is a React 19 application built with Vite 7 that shows a projects overview, a filterable run history and a run detail page with Diff, Timeline, Screenshots and Security tabs and the decision controls.

3.3.6 Supporting Module: Protect

Protect covers agents that developers still run outside the sandbox: it moves every .env value into the macOS Keychain, leaves a placeholder in the file and injects the real values only into the application started through vaultshield exec.

Figure 3.5 shows all components and the data flows between them. The CLI is the only component that talks to the Docker daemon, the dummy credential file is the only host path mounted into the sandbox, artifacts reach the backend over authenticated HTTP, and the submit command is the only component that pushes to the repository host, after confirming approval with the backend.

3.3.7 Proposed Agentic AI Orchestration

The Agentic AI Orchestrator of Objective 2 is planned for Semester VIII and has not been implemented in Semester VII; in the evaluated system the developer passes a literal prompt to vaultshield run. Its purpose is to replace a vague one-line prompt, which forces an agent to guess the framework, the files to change and the constraints, with a structured prompt, so that fewer changes are rejected at review and fewer files, APIs or packages are hallucinated. As the first row of Figure 3.2 and the context orchestrator in Figure 3.5 show, it will run on the host before any sandbox is created. A context scanner will collect a bounded, non-secret project summary: the directory listing to three levels, the start command and port detected by vaultshield init, dependency names, the start of the README and recent commit subjects. A sanitiser will replace protected values and secret-shaped strings with *** and block the request if anything secret-like remains; .env and key files are never opened. A language model will then write three to five clarifying questions in a strict JSON format, the developer will answer them at the terminal or from a file, and a prompt builder will combine the task, the project summary, the answers, a fixed Constraints section that the model cannot change, and a suggested approach. A final secret check will run before the prompt is saved as prompt.md in the run directory and passed unchanged into the existing sandboxed pipeline, where the reviewer sees it beside the diff and timeline. Because the Orchestrator sends project context to an external model and could be steered by hostile README text, it will gain no privileges: the agent still receives only dummy credentials, its change still passes trusted capture and human approval, and pattern-based sanitisation remains a stated residual risk. Its expected benefit, higher task success and approval rates with fewer hallucinated changes, will be measured on real agents in Semester VIII.

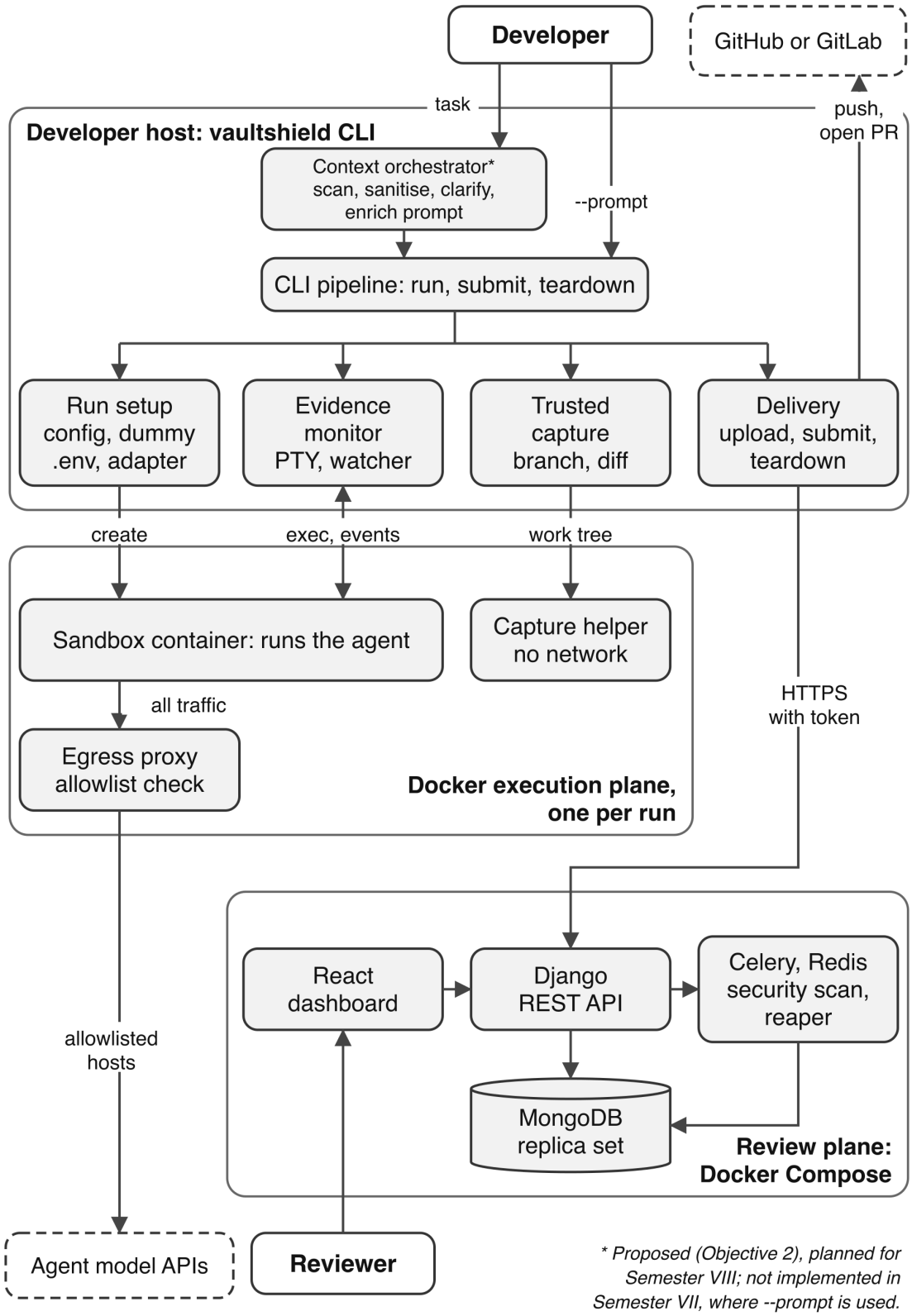


Figure 3.5: Overall system architecture

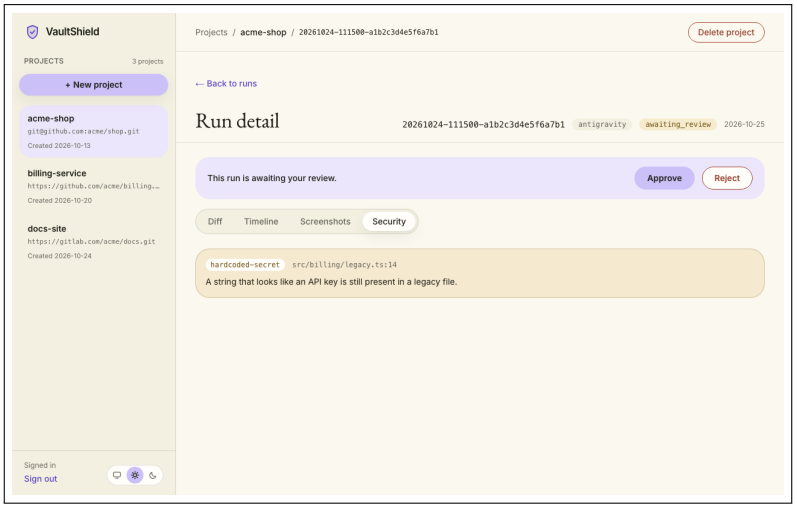


Figure 5.14: Run detail – Security tab

5.3.5 Approval Decision and Submission

When the reviewer approves a run, the buttons are replaced by the status “approved, awaiting submission” and the exact vaultshield submit command, as shown in Figure 5.15, because the backend never opens a pull request on its own.

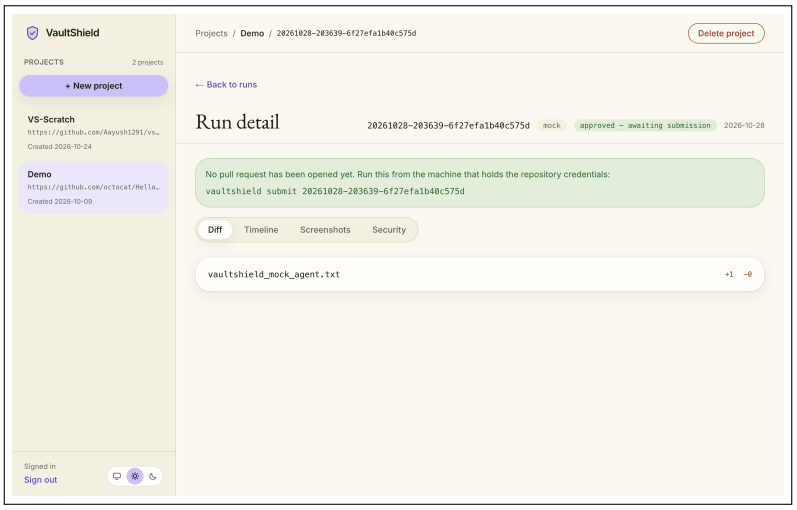


Figure 5.15: Run detail after approval, showing the submit command

5.3.6 Implementation Summary

The CLI creates the sandbox, monitors the agent, captures the change on a dedicated branch and uploads the evidence, while the dashboard presents that evidence and records the reviewer’s decision. Producing a change and accepting it therefore remain separate steps. Everything in this chapter was implemented in Semester VII: the CLI and adapters, the Docker sandbox with a fresh clone, dummy and one-shot credentials, egress control, monitoring with the event log and off-limits paths, trusted capture and the structured diff, the MongoDB backend and dashboard, security scanning, and approval-gated submission. The Agentic AI Orchestrator, stronger sandbox isolation, improved monitoring and role-based approval are planned for Semester VIII.

Two further captures are genuine executions. Capture (i) shows submit refusing a run that was still awaiting review, and no sandbox, helper or proxy left after the run. Capture (ii) is from the isolation probe of 9 October 2026, run against a sandbox in which the Antigravity agent had been signed in: the agent's credential files are inside the sandbox, direct network and Docker-socket paths fail, and the proxy records each host the agent contacted. No Antigravity task response was preserved, so none is shown.

Approval gate and cleanup check after the run, 1 Nov 2026

```
$ vaultshield submit 20261101-180424-c5458c45adecdf4a
Error: Run 20261101-180424-c5458c45adecdf4a is 'awaiting_review', not 'approved', so it cannot be su
bmitted. Nothing was pushed and no pull request was opened.
exit=1

$ docker ps -a --filter name=vaultshield-run --filter name=vaultshield-capture --filter name=vaultsh
ield-egress
NAMES     STATUS

$ docker network ls --filter name=vaultshield
NAME                 DRIVER
vaultshield_default   bridge

$ ls ../vaultshield_mock_agent.txt   # run from cli/: the host checkout root
ls: ../vaultshield_mock_agent.txt: No such file or directory
```

Execution capture (i): approval gate and cleanup check after a mock-agent run, 1 November 2026

Signed-in Antigravity agent in a VaultShield sandbox, 9 Oct 2026 (excerpt)

```
# ~/.vaultshield/probes/20261009T083128Z-egress-log-only.json (isolation_probe.py, 9 Oct 2026), sele
cted fields
agent: antigravity  egress: log-only  interactive_auth: True  https_proxy: http://vaultshield-egres
s:3128
  /root/.gemini/antigravity-cli/antigravity-oauth-token 497 bytes 0o600
  /root/.gemini/antigravity-cli/conversations/94aac82a-55f4-4de6-8658-3c7f2d5006ec.db 131072 bytes 0
o644
  /root/.gemini/antigravity-cli/log/cli-20261009_083130.log 38540 bytes 0o644
  probe internet TCP 1.1.1.1:443 -> unreachable
  probe DNS exfil (direct UDP) 8.8.8.8:53/udp -> unreachable
  probe file /var/run/docker.sock -> unreachable

# ~/.vaultshield/probes/20261009T094445Z-egress-allowlist-proxy.jsonl: first decision per host
09:44:47 deny  antigravity-unleash.goog                                    not_in_allowlist
09:44:47 deny  playwright.azureedge.net                                    not_in_allowlist
09:44:47 deny  playwright-akamai.azureedge.net                             not_in_allowlist
09:44:47 deny  playwright-verizon.azureedge.net                            not_in_allowlist
09:44:48 deny  antigravity-cli-auto-updater-974169037036.us-central1.run.app  not_in_allowlist
09:44:52 deny  play.googleapis.com                                         not_in_allowlist
09:45:03 allow oauth2.googleapis.com
09:45:04 deny  www.googleapis.com                                          not_in_allowlist
09:45:04 allow daily-cloudcode-pa.googleapis.com
... 29 decisions in total: 6 allowed, 23 denied
```

Execution capture (ii): probe and egress-proxy records from a sandbox with a signed-in Antigravity agent

5.4 Timeline Sem VII

The Gantt chart in Figure 5.16 shows the Semester VII activities across thirteen weeks, from group formation on 10 July 2026 to the external presentation and viva voce on 2 November 2026. The activities are grouped into four phases: project conception and initiation, design and implementation, evaluation and documentation, and final submission. All four members are owners of every activity, and every Semester VII activity is complete.

blocked, that a tampered in-container Git cannot hide a file from the diff, and that concurrent approve and reject requests resolve to exactly one decision.

6.2.2 System Testing

System testing drove the real vaultshield run command end to end with a real container, clone, agent process, trusted capture and teardown, and the same harness executed the 40 timed runs reported in Chapter 7.

6.3 Testing Results and Findings

Table 6.2 summarises the results of the three automated suites.

Table 6.2: Automated test suite results

Suite	Tests	Passed	Failed	Pass rate
CLI	2,080	2,078	2	99.90%
Backend	341	341	0	100.00%
Frontend	283	283	0	100.00%
Total	**2,704**	**2,702**	**2**	**99.93%**

No test was skipped. The lowest text contrast was 4.77:1 in the light theme and 5.80:1 in the dark theme, above the 4.5:1 threshold, and the layout report recorded no horizontal overflow across 63 page and width combinations. Table 6.4 presents representative test cases, each corresponding to an automated test or to the benchmark harness.

Table 6.4: Sample test cases

ID	Description	Expected and actual result	Status
TC-01	Sandbox isolation	Probing agent cannot read the host .env, home directory or Docker socket; host checkout untouched in 40 of 40 runs	Pass
TC-02	Secret suppression	No generated dummy value contains its real value; 135 of 135 tests passed	Pass
TC-03	Network blocking	CONNECT to unlisted hosts, IP literals and the host answered 403; direct DNS lookups fail	Pass
TC-04	Off-limits enforcement	Write to a protected path records a violation event, then fails the run	Pass
TC-05	Tamper resistance	A file hidden by a tampered in-container Git still appears on the run branch	Pass
TC-06	Token hygiene	All five credential-leak vectors fail the run and quarantine the diff	Pass
TC-07	Approval gating	submit refuses every non-approved run with no push and no PR	Pass
TC-08	Concurrent decisions	Exactly one winner per trial and exactly one Approval record	Pass
TC-09	Monitoring completeness	Every agent write in the timeline; present in 32 of 40 runs, while the diff recorded 40 of 40	Partial
TC-10	Hanging sign-in is bounded	Expected to stop within 60 s; took 600.3 s	Fail

6.4 Sample Test Cases

Two of the 2,704 tests failed. Both are Docker-backed tests of the optional interactive sign-in transport, used only when the antigravity agent runs with --interactive-auth; the first is TC-10 in Table 6.4.

Failure 1: interactive sign-in timeout. The hanging sign-in test expects the CLI to give up within 60 s of a silent sign-in, but it took 600.33 s. The deadline is a socket timeout on the Docker exec stream, and on Docker Desktop 29.2.1 it does not interrupt the blocking read. This is a genuine availability defect in the optional authentication flow; it exposes no secret. The correction is an independent watchdog deadline that kills the exec.

Failure 1, excerpt of pytest output

```
$ pytest --durations=2 tests/test_sandbox_execution.py::<hanging sign-in test>
___ test_a_hanging_signin_hits_the_read_timeout_instead_of_blocking_forever ____
>           assert outcome is not None
E           assert None is not None
tests/test_sandbox_execution.py:1588: AssertionError
600.33s call     tests/test_sandbox_execution.py::test_a_hanging_signin_hits_the_read_timeout_instea
d_of_blocking_forever
FAILED tests/test_sandbox_execution.py::test_a_hanging_signin_hits_the_read_timeout_instead_of_block
ing_forever
```

Failure 1: hanging sign-in test, excerpt of the pytest output (1 November 2026)

Failure 2: redacted echo expectation. The stdin test expects the raw sentinel value in the terminal output, which shows RECEIVED:[***]. Redaction replaces only an exact match of the sent value, so the output proves that the value was delivered and then masked as designed. The failure is an outdated assertion, not failed credential delivery; the assertion must expect the redacted output.

Failure 2, excerpt of pytest output

```
$ pytest --durations=2 tests/test_sandbox_execution.py::<stdin delivery test>
______ test_exec_interactive_really_delivers_stdin_to_a_container_process ______
                handle.send_line(_STDIN_SENTINEL)
>               assert f"RECEIVED:[{_STDIN_SENTINEL}]" in joined, joined
E               AssertionError: ***
E                 RECEIVED:[***]
E               assert 'RECEIVED:[interactive-stdin-sentinel-b41c]' in '***\nRECEIVED:[***]'
tests/test_sandbox_execution.py:1230: AssertionError
0.21s call     tests/test_sandbox_execution.py::test_exec_interactive_really_delivers_stdin_to_a_con
tainer_process
FAILED tests/test_sandbox_execution.py::test_exec_interactive_really_delivers_stdin_to_a_container_p
rocess
```

Failure 2: stdin delivery test, excerpt of the pytest output (1 November 2026)

Reliability impact. The two failures represent about 0.07% of the 2,704-test campaign. One is a genuine availability defect in an optional authentication flow; the other is an outdated test expectation. Neither affects the default non-interactive workflow, and the security-critical isolation, credential substitution, egress, trusted capture, token-hygiene and approval tests passed.

6.5 Conclusion on Testing

Of 2,704 automated tests, 2,702 passed, and every security-critical behaviour was exercised against real Docker and MongoDB. The remaining gaps, the sign-in deadline and the file watcher's missed events, affect availability and supplementary evidence, not isolation or change capture.